Assignment 06 Report

Course

CSE323: Advanced Embedded Systems Design
CSE411/CSE411s: Real-Time and Embedded Systems Design

Assignment

Assignment 06: Counting Semaphore for Shared Resource Management

Objective

The purpose of this assignment is to demonstrate how a counting semaphore can be used to manage a limited pool of shared resources in a FreeRTOS system. The required proof is that multiple worker tasks may compete for the resources, but no more than the available number of resources can be used at the same time.

System Scenario

The implemented design models a resource pool with 5 worker tasks competing for access, 2 identical shared resources available at any time, and 1 counting semaphore initialized with a maximum count of 2 and an initial count of 2. Each worker task repeatedly attempts to take a token, marks itself as active, simulates resource usage using `vTaskDelay()`, and releases the token afterward.

Verification Variables

The debugger watch window used `gActiveResources`, `gMaxObservedResources`, and `gTaskUsingResource[0..4]`. These variables are sufficient to show how many resources are occupied and which tasks are currently inside the resource-usage section.

Debug Method

The project was run in Keil debug mode with breakpoints at `gActiveResources++;` and `gActiveResources--;`. This allowed observation of both acquisition and release states while the five worker tasks competed for the two available tokens.

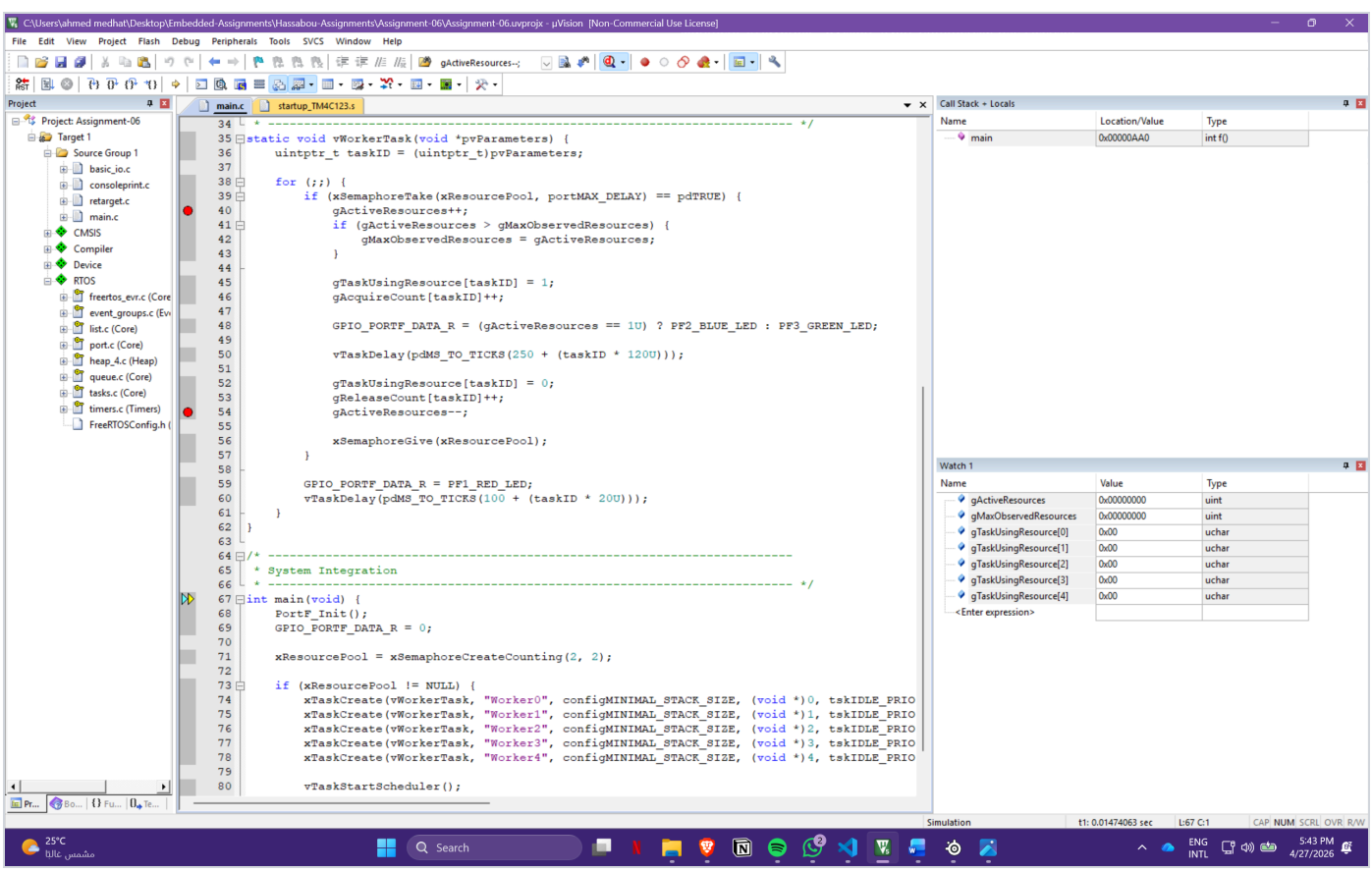
Screenshot Verification and Analysis

The alternating states in the screenshots are expected and correct. Since five worker tasks are repeatedly competing for only two resources, the exact worker IDs holding resources will change over time. What matters is that the total number of active resource users never exceeds 2.

Screenshot 1: Initial baseline before acquisition

File: `Screenshot 2026-04-27 174352.png`

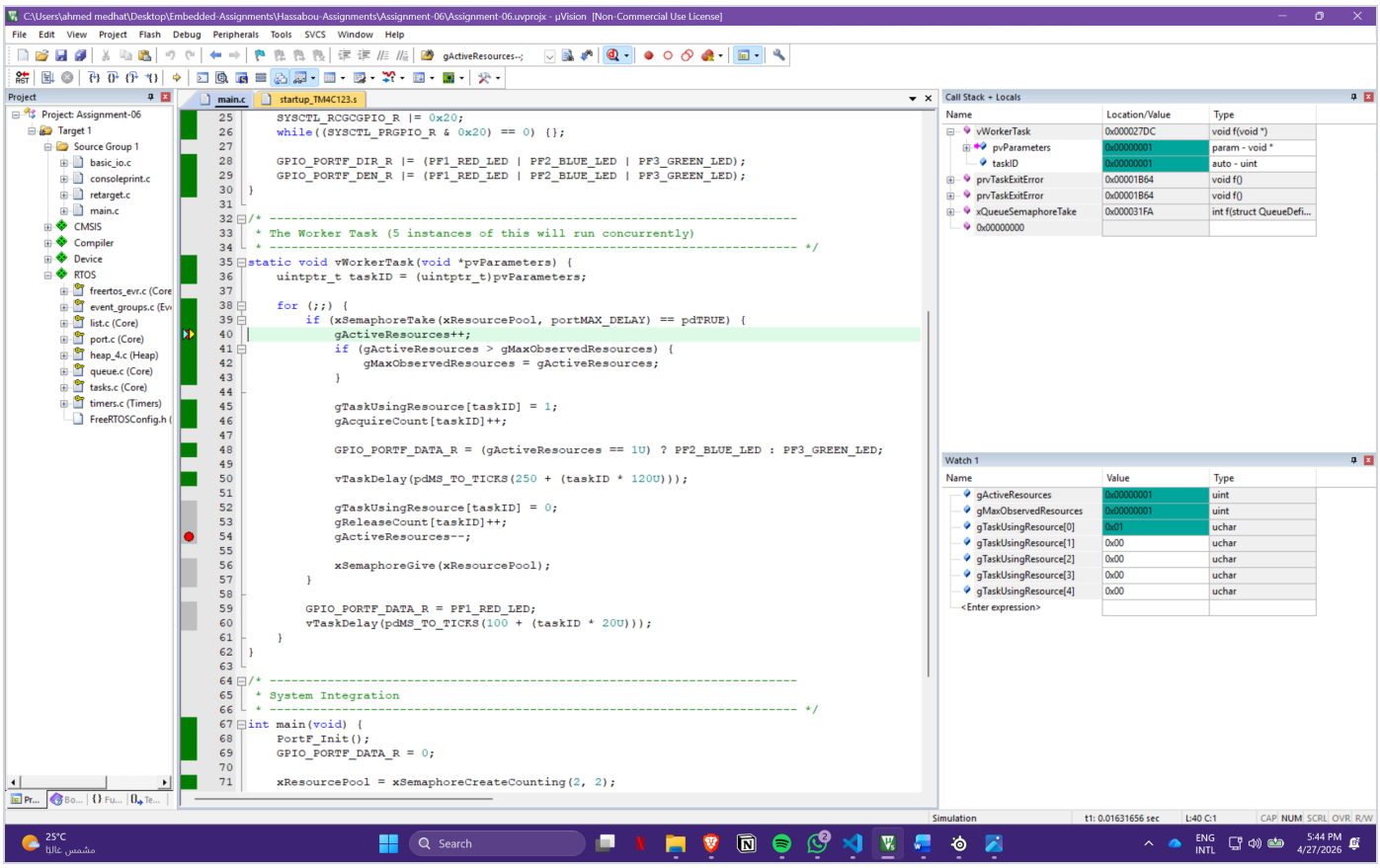
This screenshot shows the baseline state before any worker has successfully entered the resource section. At this point, `gActiveResources = 0`, `gMaxObservedResources = 0`, and all `gTaskUsingResource[]` entries are `0`. This is a valid starting state for the experiment.



Screenshot 2: First worker acquires a resource

File: `Screenshot 2026-04-27 174415.png`

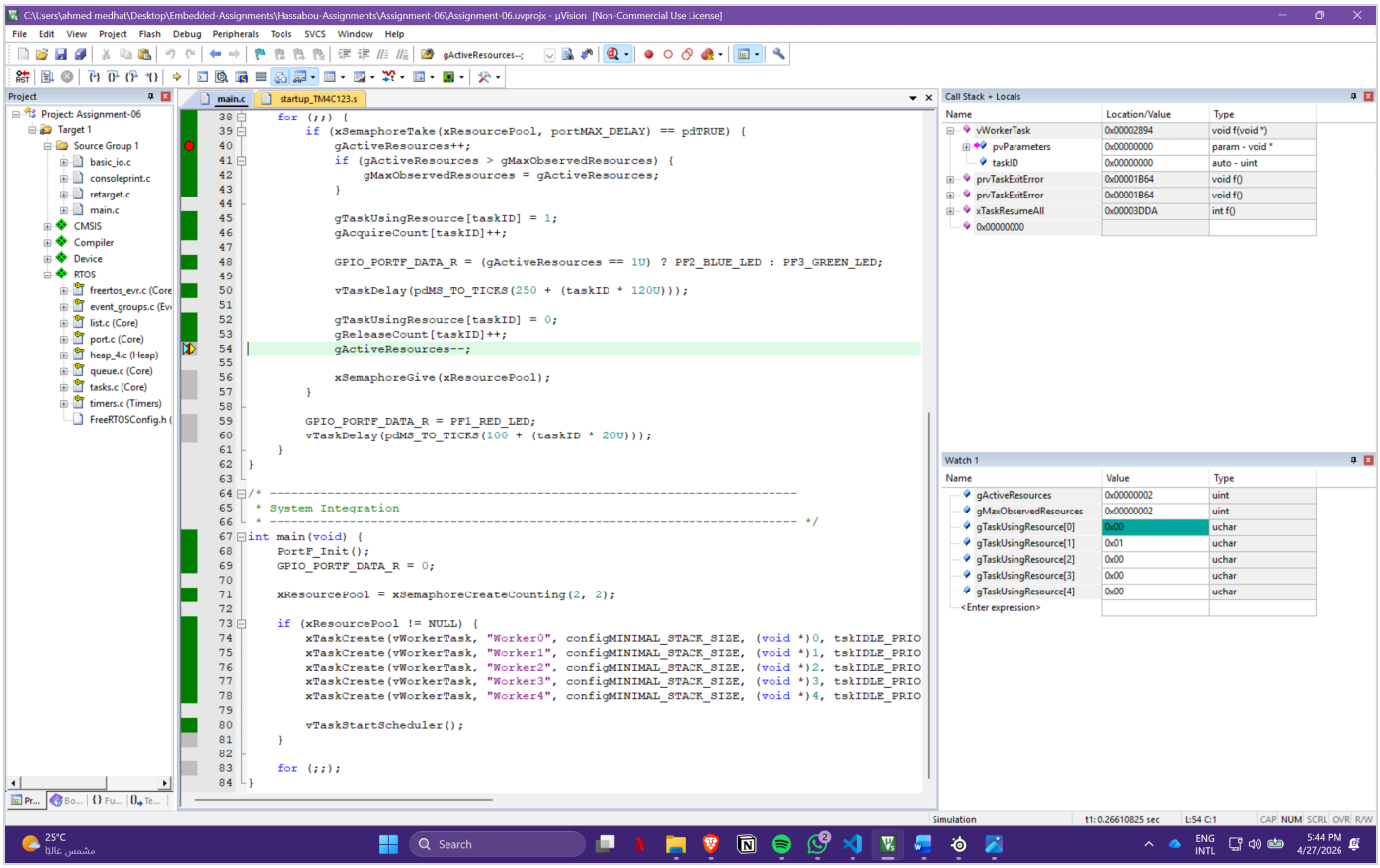
This screenshot shows the system stopped at the acquisition breakpoint. The watch values show `gActiveResources = 1`, `gMaxObservedResources = 1`, and one task entry in `gTaskUsingResource[]` is active. This proves the first worker successfully claimed one of the two resource tokens.



Screenshot 3: Two resources are in use simultaneously

File: `Screenshot 2026-04-27 174422.png`

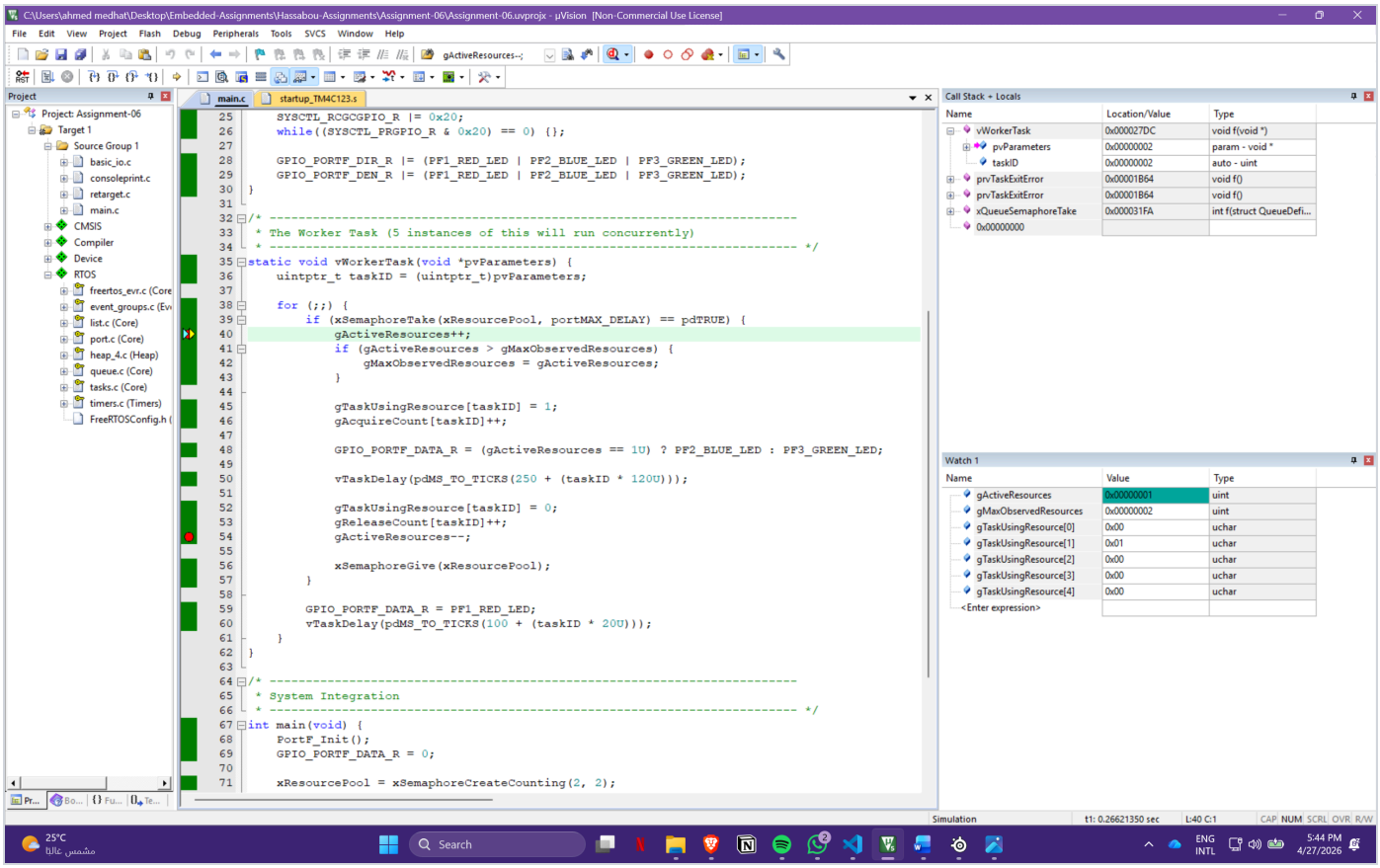
This screenshot shows the system after another worker has entered the resource section. The watch values show `gActiveResources = 2` and `gMaxObservedResources = 2`. This is the key proof that the system allows up to two concurrent resource users, matching the semaphore configuration.



Screenshot 4: Resource release while maximum remains bounded

File: `Screenshot 2026-04-27 174428.png`

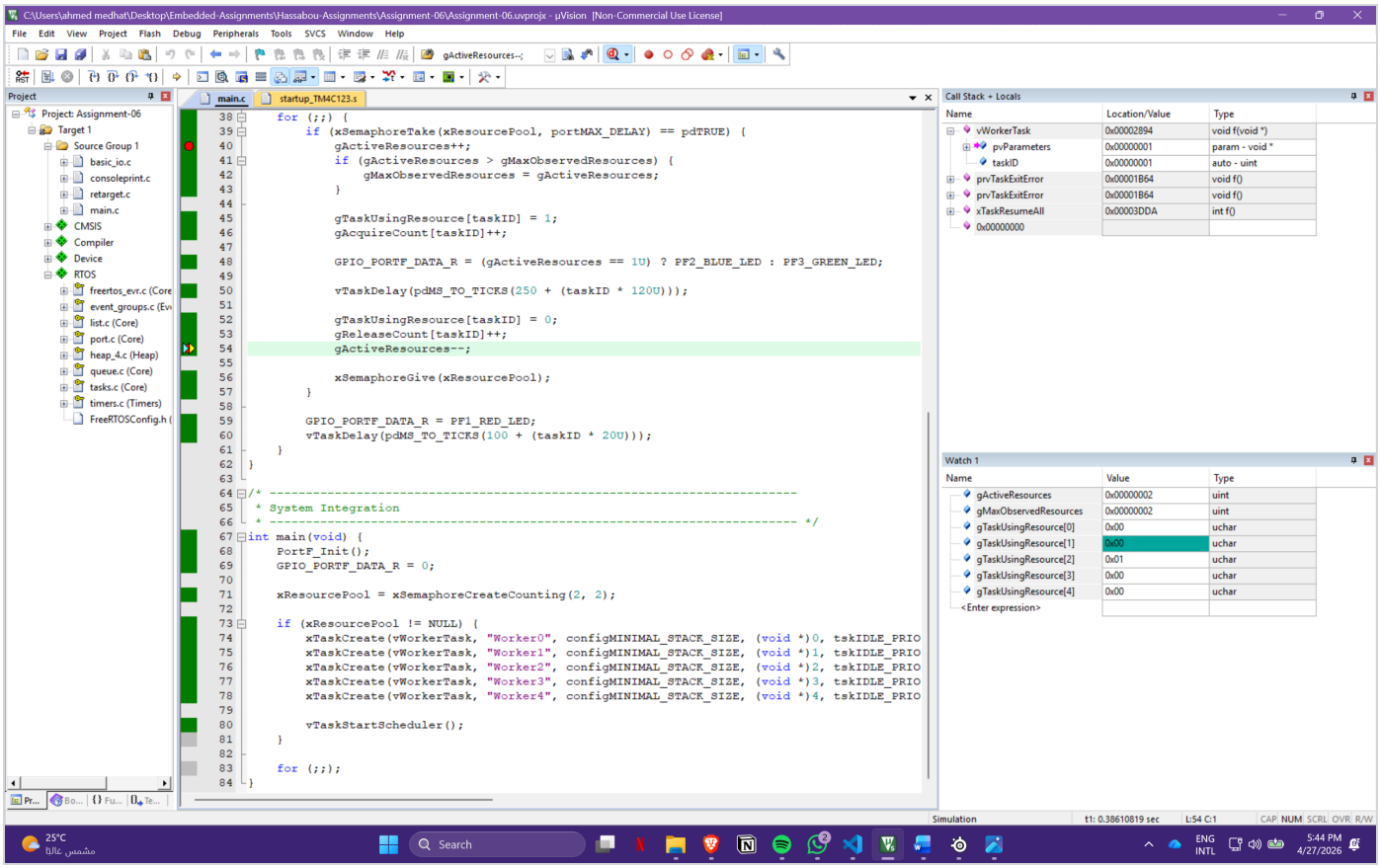
This screenshot shows the system at the release breakpoint. The watch values show `gActiveResources = 1` and `gMaxObservedResources = 2`. This means one worker released a token, but the maximum observed simultaneous usage remained at 2. This is expected and proves that the system dynamically allocates and releases resources while preserving the upper limit.



Screenshot 5: Alternating active worker identity, same resource limit

File: `Screenshot 2026-04-27 174431.png`

This screenshot shows that the identity of the active worker changed, which is normal because multiple tasks are competing continuously. The watch values still show that `gActiveResources` does not exceed `2` and `gMaxObservedResources = 2`. This is valid proof that the active worker IDs alternate over time, but the total number of simultaneous resource users never exceeds the configured limit.



Experimental Result Summary

The captured execution confirms the expected behavior of the counting semaphore resource pool: worker tasks enter and leave the resource section over time, the identity of the active worker changes from one breakpoint to another, `gActiveResources` rises and falls dynamically, `gMaxObservedResources` reaches `2` and never exceeds it, and the counting semaphore therefore limits access to the shared resource pool to two concurrent users.

Conclusion

The assignment objective was achieved successfully. The implemented FreeRTOS application demonstrates that a counting semaphore can be used to manage a limited pool of two shared resources among five competing worker tasks. The debugger evidence shows that while the active task identities alternate over time, the total number of simultaneous resource users never exceeds 2. This confirms correct counting semaphore behavior for resource management.